The final move ❻ transfers ownership of the thread running `some_function` back to `t1` where it started. But in this case `t1` already had an associated thread (which was running `some_other_function`), so `std::terminate()` is called to terminate the program. This is done for consistency with the `std::thread` destructor. You saw in section 2.1.1 that you must explicitly wait for a thread to complete or detach it before destruction, and the same applies to assignment: you can't just "drop" a thread by assigning a new value to the `std::thread` object that manages it.

The move support in `std::thread` means that ownership can readily be transferred out of a function, as shown in the following listing.

Listing 2.5. Returning a `std::thread` from a function

```
std::thread f()
{
    void some_function();
    return std::thread(some_function);
}
std::thread g()
{
    void some_other_function(int);
    std::thread t(some_other_function,42);
    return t;
}
```

Likewise, if ownership should be transferred into a function, it can just accept an instance of `std::thread` by value as one of the parameters, as shown here:

```
void f(std::thread t);
void g()
{
    void some_function();
    f(std::thread(some_function));
    std::thread t(some_function);
    f(std::move(t));
}
```

One benefit of the move support of `std::thread` is that you can build on the `thread_guard` class from listing 2.3 and have it actually take